

SIMATS ENGINEERING

C05 – ASSESSMENT TASK 2 | ERROR ANALYSIS TASK

SET 1 – ALL 5 QUESTIONS ANSWERED

The answers below are based on the error scenarios given in the uploaded question paper. Each response identifies the likely design error and proposes corrective actions.

1. Uneven Input Splits and Idle Reducers

Problem: The Hadoop job repeatedly fails because input splits are uneven and some reducers remain idle. **Likely design error:** The input data is partitioned poorly, creating highly uneven workloads. One or more partitions may contain much more data than others. This creates a data-skew problem: some mapper/reducer tasks finish quickly while another task processes a much larger partition. Reducers may therefore remain idle while waiting for the overloaded task. **Corrective actions:** • Review input split sizing and ensure that files are divided into reasonably balanced splits. • Use an appropriate input format and block/split configuration for the dataset. • Check the partitioner so that intermediate keys are distributed evenly among reducers. • Identify hot keys or skewed data and use techniques such as better key partitioning or salting when appropriate. • Increase reducer parallelism only after fixing the underlying data-skew problem. **Result:** Balanced partitions and better key distribution allow mapper and reducer tasks to work more evenly, improving cluster utilization and reducing job completion time.

2. HDFS Data Unavailability after a Single Node Failure

Problem: An HDFS deployment experiences frequent data unavailability after one node fails. **Probable design error:** The most likely issue is insufficient block replication or poor placement of replicas. If a data block has only one available copy, failure of the DataNode holding that block makes the data unavailable. A related issue could be an improperly designed or unavailable NameNode, because the NameNode maintains HDFS metadata required to locate blocks. **Corrective actions:** • Configure an appropriate HDFS replication factor so important blocks have multiple copies. • Ensure replicas are placed on different DataNodes and, where applicable, different racks. • Monitor under-replicated blocks and allow HDFS to recreate missing replicas on healthy nodes. • Use NameNode high availability with active/standby NameNodes to avoid a single metadata failure point. • Regularly monitor DataNode and NameNode health. **Result:** Proper block replication and NameNode high availability ensure that a single node failure does not make required data unavailable.

3. Duplicate Records in ETL and Apache NiFi Ingestion

Problem: An ETL and Apache NiFi ingestion workflow creates duplicate records in the analytics store. **Likely causes:** The pipeline may process the same input more than once because of retries, repeated file ingestion, processor re-execution, or missing deduplication. Another possibility is that the destination accepts repeated writes without checking whether a record has already been processed. **Corrective actions:** • Assign a unique record ID or business key to every incoming record. • Configure NiFi flow logic so already-processed files or records are not ingested repeatedly. • Use suitable NiFi provenance and state-management mechanisms to track processing state. • Make the destination write operation idempotent, so repeating the same operation does not create another copy. • Add a deduplication step before loading the analytics store. • Handle retries carefully so a failed acknowledgement does not cause an already-successful record to be inserted again. **Result:** Combining unique identifiers, state tracking, deduplication and idempotent loading prevents duplicate records while still allowing safe retries.

4. YARN Resource Starvation with Multiple Users

Problem: A YARN cluster suffers from resource starvation when several users submit jobs simultaneously. **Likely scheduling error:** The cluster may be using an unsuitable scheduling configuration, with no effective resource queues, capacity limits, priorities or fair allocation between users. One user's workload can consume most of the available CPU and memory, leaving insufficient resources for other jobs. **Corrective actions:** • Configure YARN queues to separate workloads and users. • Apply capacity or fair-scheduling policies so resources are shared according to organizational requirements. • Set queue-level CPU and memory limits where appropriate. • Configure application priorities and maximum resource limits to prevent one application from monopolizing the cluster. • Monitor queue utilization and adjust resource allocations as workloads change. • Reserve appropriate resources for critical production workloads. **Result:** Proper queue-based resource management prevents a single user or application from exhausting cluster resources and provides predictable access for multiple users.

5. Backup Restores Outdated Data after Recovery

Problem: A distributed-storage backup strategy restores outdated data after a recovery operation. **Probable design flaw:** The backup or replication process is not maintaining an accurate, current recovery point. The system may be using stale backups, infrequent snapshots, incorrect replication synchronization, or a recovery procedure that selects an older backup without verifying its timestamp or consistency. **Corrective actions:** • Define appropriate Recovery Point Objectives (RPO) and Recovery Time Objectives (RTO). • Schedule backups or snapshots frequently enough to meet the required RPO. • Track backup versions and timestamps clearly. • Verify that replication is synchronized and monitor replication lag. • Test backup restoration regularly instead of assuming backups are valid. • Use checksums or integrity validation to confirm that recovered data is complete and consistent. • During recovery, select the newest verified backup or snapshot that satisfies the recovery requirement. **Result:** Version-aware backups, monitored replication, integrity checks and regular restore testing reduce the chance of recovering stale data and improve disaster recovery reliability.

Quick interview structure: For each error-analysis question, state the problem, identify the likely design error, give 3–5 corrective actions, and conclude with the expected improvement.